

IA além do teste de software

1. Contexto e visão original (2015)

- Conceito de "right to left thinking": imaginar o futuro desejado e trabalhar de trás para frente.
- Em 2015, o teste era fragmentado e muito manual, inclusive a automação, baseada em scripts escritos à mão.
- Junto com Jason Arbon (test.ai), formulou a visão de "lights out testing": testes com pouca ou nenhuma intervenção humana, inspirada em fábricas totalmente robotizadas.
- Essa visão se dividia em dois eixos: IA para testar software e testar sistemas de IA.
- Na test.ai, a IA era usada em:
 - Testes de apps web/UI em escala global (lojas de apps)
 - Visão computacional e aprendizado por reforço para testar videogames (um dos problemas mais difíceis de teste)
 - Qualidade de vídeo
- Previsão da época: a IA tomaria conta de testes de UI, API e unitários, depois segurança, desempenho e design. Ainda assim, o teste continuaria central, com sistemas de IA capazes de "se auto testar" ao longo do tempo.

2. O momento atual: IA generativa (2024)

- A transformação pareceu gradual e depois súbita (já se usava GPT-3 na test.ai).
- A IA gera imagens, áudio, vídeo, arte e música, tornando cada vez mais difícil distinguir o real do falso, o que traz preocupações de segurança.
- ChatGPT: seu grande mérito foi tornar a IA acessível a todos, colocando o PLN (processamento de linguagem natural) nas mãos de qualquer pessoa.

3. Casos de uso de IA generativa em teste (já mainstream)

- Projeto de casos de teste: analisar requisitos, código e APIs; cobrir limites e classes de equivalência; com modelos multimodais, analisar diagramas de máquina de estados e tabelas de decisão.

- Implementação: gerar testes unitários, de integração e scripts de UI; gerar testes que percorrem caminhos específicos do código.
- Documentação e manutenção: manter casos de teste atualizados e reduzir redundância.
- Dados de teste sintéticos: gerar dados realistas a partir de relacionamentos de dados reais.
- Execução e análise: resumir e interpretar grande volume de falhas e defeitos.
- Conclusão: o teste sempre ficou devendo em inovação e automação, e agora mais coisas podem ser automatizadas.

4. Produtividade = vazão + qualidade + eficiência

- Produtividade não é só velocidade: andar rápido com erros resulta em lentidão no fim.
- A adoção da IA generativa é impulsionada pela interseção de três fatores: throughput, qualidade e eficiência.
- Visão holística do ciclo de vida (SDLC): requisitos, design, arquitetura, código, teste, implantação.
- Base obrigatória: excelência em engenharia, práticas ágeis, shift left e engenharia de qualidade. Sem boas práticas, a IA só leva a "lugar nenhum rapidamente".

5. Ferramentas citadas ao longo do ciclo de vida

Etapa	Ferramenta	O que faz
Requisitos	AI Genie (Jira/Atlassian, da EPAM, gratuita)	Gera critérios de aceitação e diagramas de sequência a partir de linguagem natural
Design de UI	Wizard (comercial)	Gera protótipos de apps a partir de um prompt
Arquitetura	DiagramGPT (eraser.io)	Gera diagramas de arquitetura e sequência; detecta o tipo de diagrama necessário
Código	GitHub Copilot	Gera blocos inteiros de código na IDE

Teste na IDE	Alita (EPAM)	Prompts dentro da IDE para gerar testes unitários conforme a linguagem do projeto
CI/CD	IA no pipeline	Gera Terraform, trata erros em pull requests, sugere correções, atua em quality gates (go/no-go)

- Filosofia: levar a IA para dentro das ferramentas do dia a dia, evitando troca de contexto (em vez de uma janela isolada de chat).

6. Qualidade = Produtividade

- Mantra da Ultimate Software: se a qualidade sobe, a produtividade sobe; se cai, a produtividade cai.
- Engenharia de qualidade: construir qualidade no produto durante o processo, não testá-la só no final (contra o modelo cascata).
- Verificação e validação em cada etapa são mais eficientes.
- A IA está levando à automação da engenharia de qualidade.

7. Desafios para adotar IA nas organizações

- Segurança e propriedade intelectual: o primeiro obstáculo; sem resolvê-lo, não há avanço.
- Capacitação das pessoas: perguntas erradas geram respostas erradas.
- Alucinações: conteúdo que parece correto, mas pode estar errado.
- Medição de produtividade e qualidade: é muito difícil.

8. Pilares da solução

- Acesso seguro e corporativo à IA: usar alternativas como Azure ou Google em vez da API pública; a EPAM criou o AI DIAL, um proxy seguro que dá acesso a vários modelos (Claude, GPT-4, Gemini) dentro de seu ambiente.
- Cultura e colaboração: a Alita evoluiu para uma plataforma de compartilhamento de prompts entre equipes e projetos, rumo a sistemas mais automatizados.
- Engenharia de prompts:

- Há muito hype ("será uma grande carreira"), mas é habilidade necessária, ainda que não vire profissão por si só.
- Certificação para a comunidade de teste, criada com a AI United e a Bright Test, com foco em formar formadores e escalar globalmente.
- Testar sistemas de IA: para confiar na IA, é preciso poder testá-la.

9. Como testar sistemas de IA

- Série de artigos: "AI shift left, ainda testando à direita (shift right)".
- Envolve muitos papéis: engenheiros de ML, ciência de dados, segurança, engenharia de software, usuário final.
- Perguntas antes de construir:
 - Isso realmente precisa de IA?
 - Que dados foram usados?
 - Que métricas definem bom comportamento?
 - Usa RAG (geração aumentada por recuperação)?
- Testes após a implantação são indispensáveis, porque esses sistemas são dinâmicos e não determinísticos (mesma pergunta, respostas diferentes).
Tipos: IA adaptativa e IA generativa.
- Iceberg de teste de IA (o que existe abaixo da experiência do usuário):
 - Integridade do conteúdo (precisão e coerência)
 - Engajamento (fluência, compreensão do contexto)
 - Responsabilidade e justiça
 - Segurança e proteção
 - Utilidade para a tarefa (resultados concisos ou prolixos)
- IA confiável exige:
 - Controlabilidade e observabilidade
 - IA explicável (XAI): dados, previsões e algoritmos explicáveis, para permitir depuração e melhoria

10. O futuro: rumo à autonomia

- Exemplos de tendência: Devin AI e CoTester (mencionados sem endosso).
- Evolução em "ondas":
 1. IntelliSense
 2. Copilotos

3. Agentes como assistentes (estágio atual)
 4. Agentes como executores primários
 5. Autonomia total (pergunta em aberto: e se se tornarem superinteligentes? qual o papel dos humanos?)
- Ainda não há autonomia plena, mas os avanços aproximam esse cenário.

11. Mensagem final: o papel da comunidade de teste

- Não basta esperar que a IA melhore sozinha: ela será usada para o bem e para o mal.
- Os testadores devem contribuir com:
 - Estruturas éticas e IA responsável
 - Governança dos sistemas de IA
 - Explicabilidade e transparência
- Esse será o futuro do testador de software.
- A visão vai além da automação: trata-se da segurança e da proteção das pessoas.